

Cafe Sales Data Cleaning & Dashboard Documentation

Data wrangling, exploratory analysis, and Power BI dashboard build for a raw point-of-sale export.

Prepared by Urbanus | Data Science Instructor & Freelance Data Analyst, Nairobi, Kenya

1. Project Overview

This document describes the end-to-end process of taking a raw, intentionally messy cafe point-of-sale export and turning it into a cleaned, analysis-ready dataset and an interactive Power BI dashboard. It covers the state of the raw data, every cleaning decision made and the reasoning behind it, the final dataset structure, exploratory findings, and the dashboard build itself.

The intent of this write-up is to make the project reviewable by someone who was not involved in building it: what problems existed in the source file, what was done about each one, why that specific method was chosen over the alternatives, and what the data supports once cleaned.

1.1 Source Data

Attribute	Detail
File	dirty_cafe_sales.csv (raw export)
Raw shape	10,000 rows x 8 columns
Final cleaned shape	9,469 rows x 8 columns
Rows removed	531 rows (5.31%) — unrecoverable after imputation, see Section 4.6
Time period covered	1 January 2023 to 31 December 2023
Tools used	Python (pandas, numpy), ydata-profiling, matplotlib/seaborn/plotly for EDA, Power BI for the dashboard

2. Dataset Description

The raw file contained eight columns, all read in by pandas as generic text (object dtype), which was itself the first data quality signal, since three of the eight columns are inherently numeric or date-based and should not have loaded as text.

Column	Expected type	Description
Transaction ID	Text	Unique identifier per transaction, e.g. TXN_1961373. Complete in the raw file, 0 missing.
Item	Text (category)	Menu item sold: Coffee, Cake, Cookie, Salad, Smoothie, Tea, Sandwich, Juice.
Quantity	Numeric	Units sold in the transaction.

Column	Expected type	Description
Price Per Unit	Numeric	Menu price per unit for that item.
Total Spent	Numeric	Total transaction value. Expected to equal Quantity x Price Per Unit.
Payment Method	Text (category)	Cash, Credit Card, or Digital Wallet.
Location	Text (category)	In-store or Takeaway.
Transaction Date	Date	Calendar date of the transaction.

3. Initial Data Quality Assessment

Before any cleaning, the raw file was profiled to establish exactly what was wrong with it rather than assuming. Two categories of problems were found.

3.1 Placeholder strings instead of real nulls

Two literal string values, "ERROR" and "UNKNOWN", were present across multiple columns in place of missing data. Because these are valid-looking strings rather than pandas-recognized nulls (NaN), they would not be caught by a standard `.isnull()` check and would silently corrupt any numeric or categorical analysis if left in place, for example, "ERROR" sitting inside the Total Spent column would prevent that entire column from being summed.

3.2 Missing values, once placeholders were standardized to NaN

Column	Missing count	Missing %
Item	969	9.7%
Quantity	479	4.8%
Price Per Unit	533	5.3%
Total Spent	502	5.0%
Payment Method	3,178	31.8%
Location	3,961	39.6%
Transaction Date	460	4.6%

Payment Method and Location carried by far the heaviest gaps, close to a third and two-fifths of all rows respectively. This points to a Point-of-Sale data capture problem rather than random noise, and shaped the imputation strategy chosen for those two columns in Section 4.4.

4. Data Cleaning Methodology

Cleaning was done as a sequence of targeted fixes, each one re-verified against the remaining missing-value count before moving to the next, rather than one blanket operation. The order matters: numeric columns were recovered using the business relationship $\text{Total} = \text{Quantity} \times \text{Price}$ before any rows were dropped, so that dropping only happened once every recoverable value had actually been recovered.

4.1 Standardizing missing value placeholders

"ERROR" and "UNKNOWN" were replaced with proper NaN values across the entire dataframe using `df.replace()`. This was a prerequisite step: it converts the messy data into a single consistent representation of "missing" so every later step, from type conversion to null counting, behaves correctly.

4.2 Data type conversion

Quantity, Price Per Unit, and Total Spent were converted to numeric using `pd.to_numeric(..., errors='coerce')`, and Transaction Date was converted using `pd.to_datetime(..., errors='coerce')`. The `errors='coerce'` argument tells pandas to convert anything it cannot parse into NaN instead of raising an exception and stopping the script, which is what allowed the placeholder cleanup in 4.1 to surface as proper missing values here rather than as parsing errors.

4.3 Recovering numeric values using business logic

Rather than immediately imputing with a generic statistic, the relationships that must hold true in this dataset (every item has a fixed menu price, and Total Spent is always $\text{Quantity} \times \text{Price}$) were used to recover values with certainty before resorting to statistical estimates.

- **Price Per Unit from Item:** each item's most frequent (mode) price was computed and used to fill missing prices for that item, e.g. every missing price for a Coffee row was filled with \$2.00. This reduced Price Per Unit's missing count from 533 to 54.
- **Item from Price Per Unit:** a reverse lookup was built, but only for prices that map to exactly one item. If a price point is shared by two menu items (e.g. two items both priced at \$3.00), that price is not a reliable signal of which item it was, so it was deliberately excluded from the lookup rather than guessed. This is a conservative choice: it recovers fewer rows than a naive lookup would, but every row it does recover is correct.
- **Total Spent and Quantity from each other:** missing Total Spent was filled with $\text{Quantity} \times \text{Price Per Unit}$, and missing Quantity was filled with $\text{Total Spent} / \text{Price Per Unit}$, using whichever of the two values was still present. This is exact recovery, not estimation, since the relationship is a fixed business rule, not a statistical pattern.

4.4 Imputing Payment Method and Location

These two columns had the heaviest missing rates (31.8% and 39.6%) and no logical relationship to any other column that could recover them with certainty, so a statistical imputation method had to be chosen. Two options were considered:

- **Mode imputation** (fill every missing value with the single most common category) was rejected. With missing rates this high, forcing roughly a third to two-fifths of the dataset into one category would have artificially inflated that category and distorted any downstream question like "what

share of customers pay by cash," since the answer would then partly reflect the imputation method rather than real customer behavior.

- **Random distribution-based imputation** was used instead: missing values were filled by sampling from the existing (non-missing) category proportions, so if the observed split were roughly 33/33/33 across payment methods, the missing rows were filled following that same split rather than collapsing to one value. This preserves the shape of the original distribution instead of distorting it.

The same random distribution approach was applied to the 501 remaining missing Item values after the price-based recovery in 4.3, for the same reason: Item is a categorical field with no further logical recovery path available, and mode imputation would have overstated whichever item happened to be most popular.

4.5 Dropping unrecoverable rows

After every recoverable value had been filled, 23 rows were still missing Quantity, 54 still missing Price Per Unit, 23 still missing Total Spent, and 460 still missing Transaction Date, with no remaining logical or statistical basis to fill them credibly. These were dropped rather than imputed, for column-specific reasons:

- **Quantity, Price Per Unit, Total Spent:** these are numeric columns feeding every downstream financial calculation. Filling them with a placeholder like "Unknown" would silently convert the entire column back to text, breaking every sum, mean, or aggregation performed on it. At a combined ~0.7% of rows, dropping was the lower-risk choice against corrupting three core numeric columns.
- **Transaction Date:** already converted to a proper datetime type. A missing date cannot be meaningfully imputed (there is no logical or statistical basis for guessing which day a transaction happened), and any time-based analysis, such as monthly or day-of-week trends, would exclude these rows regardless of what placeholder was used, so dropping changes nothing for that analysis and avoids introducing a fabricated date.

This brought the dataset from 10,000 to 9,469 rows, a reduction of 531 rows (5.31%).

4.6 Duplicate check

The cleaned dataset was checked for fully duplicated rows using `df.duplicated()`. None were found (0 duplicate rows), so no deduplication step was required.

4.7 Outlier detection and validation

The IQR (interquartile range) method was applied to Quantity, Price Per Unit, and Total Spent. Quantity and Price Per Unit returned zero outliers. Total Spent flagged 259 rows (2.7% of the cleaned data) as statistical outliers, all above the upper IQR bound of \$24.00.

Rather than removing or capping these automatically, they were investigated individually before deciding what to do with them. Every flagged row was a transaction of 5 units of Salad at \$5.00 per unit, totaling exactly \$25.00, which is the mathematically maximum possible transaction value given this cafe's real menu (5 units at the highest menu price of \$5.00). Because no flagged value exceeded that ceiling, none of the 259 rows represented an impossible or corrupted value, they are simply large-but-legitimate orders.

Decision: all 259 flagged rows were retained. A statistical outlier is not automatically a data error; here, IQR flagged values that are unusual relative to the rest of the distribution but fully consistent with the business's actual pricing structure. Removing them would have deleted real, valid high-value transactions rather than corrected an error.

5. Final Cleaned Dataset

Column	Final dtype	Missing values
Transaction ID	object (text)	0
Item	object (text)	0
Quantity	float64	0
Price Per Unit	float64	0
Total Spent	float64	0
Payment Method	object (text)	0
Location	object (text)	0
Transaction Date	datetime64	0

Final shape: 9,469 rows x 8 columns, saved as Clean_Cafe_Sales_Data.csv. Every column now holds its correct data type, and there are zero missing values across the entire dataset.

6. Exploratory Data Analysis

With the data cleaned, the following patterns were examined: revenue concentration by item, sales trend over time, and the split across payment method and location.

6.1 Revenue by item

Salad, Sandwich, and Smoothie are the top three items by total revenue; Cookie is the lowest. The full ranking is visible on the dashboard's item chart in Section 7.

6.2 Sales trend over time

Monthly revenue is close to flat across all twelve months of 2023, and revenue by day of week is similarly close to even across Monday through Sunday. There is no material seasonal pattern, growth trend, or weekday effect visible in this dataset.

6.3 Payment method and location split

Both categorical splits are close to even rather than dominated by one category:

- **Payment method:** Credit Card 33.5%, Digital Wallet 33.5%, Cash 32.9% of revenue, a near-even three-way split.

- **Location:** In-store 50.6%, Takeaway 49.4% of revenue, effectively an even split.

A note on interpretation: because Payment Method and Location were partly filled using random distribution imputation (Section 4.4), and the imputation was deliberately designed to preserve the existing proportions, these near-even splits partly reflect that imputation choice for the ~32-40% of each column that was originally missing, in addition to whatever the true underlying pattern was. The flatness across months and days, by contrast, was not affected by any imputation method and is a fully observed pattern in the recovered data. Given how uniform every dimension of this dataset turns out to be once cleaned, business conclusions drawn from it (e.g. claiming a dominant payment method or a "grab-and-go" skew) should be stated cautiously rather than as confident findings, since the data does not show a meaningful lean in either direction.

7. Power BI Dashboard

The cleaned dataset was loaded into Power BI Desktop to build an interactive one-page report.

7.1 Data model

A supporting date table was created using `CALENDAR(DATE(2023,1,1), DATE(2023,12,31))` and related to Transaction Date, with derived Month and Day-of-Week columns, the latter sorted by a hidden numeric helper column so it displays Monday through Sunday rather than alphabetically.

7.2 Core measures (DAX)

Measure	DAX
Total Revenue	<code>SUM('CafeSales'[Total Spent])</code>
Total Transactions	<code>DISTINCTCOUNT('CafeSales'[Transaction ID])</code>
Avg Order Value	<code>DIVIDE([Total Revenue], [Total Transactions])</code>
Total Units Sold	<code>SUM('CafeSales'[Quantity])</code>

7.3 Report layout

The report is organized top to bottom: a KPI row (Total Revenue, Avg Order Value, Total Transactions, Total Units Sold), an interactive slicer row for Location and Payment Method, and a set of four charts: Total Revenue by Item, Total Revenue by Day of Week, and Total Revenue by Month, alongside the Location and Payment Method filter controls. All charts cross-filter each other by default, and y-axes were deliberately zeroed rather than auto-scaled, so that the near-flat patterns described in Section 6 are represented honestly rather than visually exaggerated.

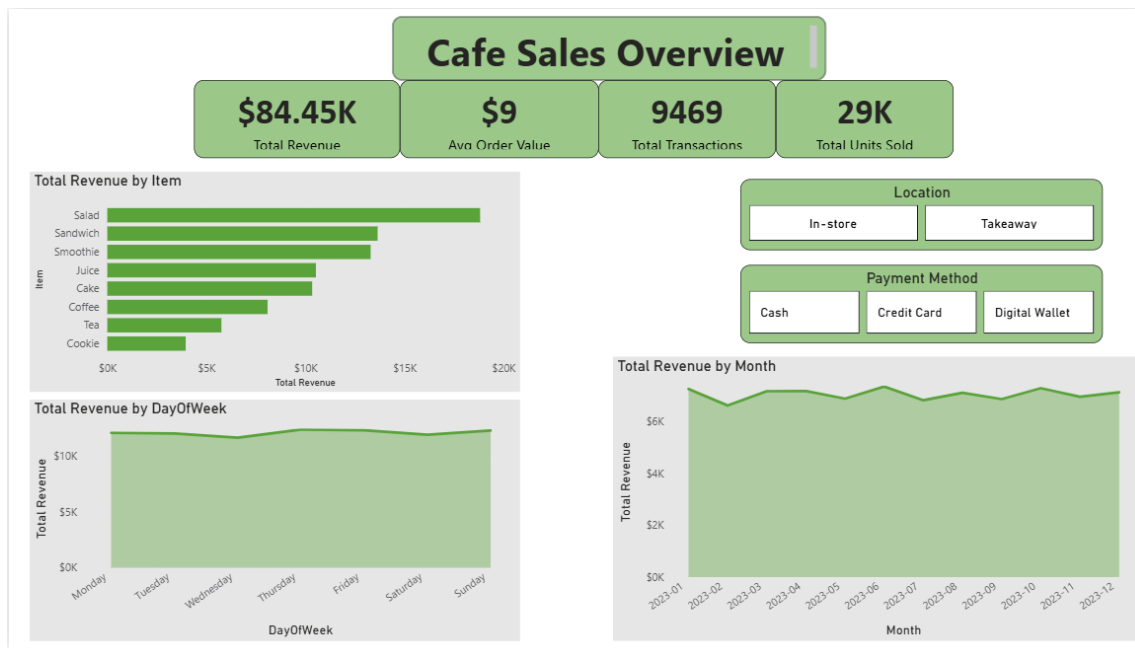


Figure 1. Final Power BI dashboard — Cafe Sales Overview.

8. Key Decisions and Rationale

Decision point	Alternative considered	Why this approach was chosen
Payment Method / Location / Item missing values	Mode imputation	Mode imputation would force ~32-40% of rows into a single category, artificially inflating it and distorting the true distribution. Random distribution imputation preserves the existing proportions instead.
Item recovery from Price	Map every price to its most common item	Only prices unique to a single item were mapped, to avoid guessing between items that share the same price point. Fewer rows recovered, but zero incorrect guesses introduced.
Rows with unrecoverable numeric fields or dates	Fill with a placeholder value	Filling numeric columns with text like "Unknown" would revert the whole column back to text type, breaking every downstream calculation. Dropping affected ~0.7% of rows for numeric fields and 4.6% for dates, a smaller cost than corrupting the columns.
259 IQR-flagged Total Spent outliers	Cap or remove as errors	Every flagged value was individually verified against the maximum mathematically possible order size and found to be a legitimate high-value transaction, not a data error, so all were retained.

9. Limitations and Notes for Reviewers

- The dataset's near-uniform distribution across payment methods, locations, months, and days of the week is consistent with synthetically generated data rather than an organically collected sales log. This should be disclosed to anyone reviewing the analysis as a portfolio piece, so the project is framed accurately as a data cleaning and dashboarding exercise rather than a business-insights case study.
- Roughly a third of Payment Method values and two-fifths of Location values were imputed rather than originally observed. Any conclusion drawn specifically from those two columns carries that caveat.
- The Power BI dashboard currently covers descriptive, filterable views of the cleaned data. It does not include predictive or forecasting elements, which were out of scope for this stage of the project.